

Ingeniería de software II

- Bibliografía básica:
- PRESSMAN, Roger S.. Ingeniería del software, un enfoque práctico. Ed.Mc Graw Hill. (2010)
- - SOMMERVILLE, Ian. Ingeniería de Requerimientos. Ed.Addison Wesley/Pearson. (2006)
- Promoción directa. Nota no menor a 8(ocho) en parciales y TPI. Sin recuperatorio.
- Regularidad: parciales y TPI aprobados.

Ingeniería de Software II

- ✓ El Software. Conceptos generales. Cualidades del Software.
- ✓ Porque la creación de software es un problema de ingeniería.

¿Qué es el software?

Es el conjunto de los programas de cómputo, procedimientos, reglas, documentación y datos asociados, que forman parte de las operaciones de un sistema de computación.

El software es un transformador de información —produce, administra, adquiere, modifica, despliega o transmite información que puede ser tan simple como un solo bit o tan compleja como una presentación con multimedios generada a partir de datos obtenidos de decenas de fuentes independientes—.

El software como producto final

- **Ingeniero:** conjunto de programas, datos y otros productos terminados que constituyen el sw de computadora.
- **Usuario:** es un producto/servicio que le otorga un valor agregado a su vida cotidiana.



¿Cuáles son los atributos de un buen software?

- Debe tener la funcionalidad y el rendimiento requeridos por el usuario.
- Fácil de mantener,
- Confiable,
- Y fácil de utilizar.

Ingeniería de Software

La noción de ingeniería del software fue propuesta inicialmente en 1968 en una conferencia para discutir lo que en ese entonces se llamó la «crisis del software».

Esta crisis del software fue el resultado de la introducción de las nuevas computadoras basadas en **circuitos integrados**. Su poder hizo que las aplicaciones hasta ese entonces irrealizables fueran una propuesta factible. El software resultante fue de órdenes de magnitud más grande y más complejo que los sistemas de software previos.

Ingeniería de Software

La experiencia previa en la construcción de estos sistemas mostró que un enfoque informal para el desarrollo del software no era muy bueno.

Los grandes proyectos a menudo tenían años de retraso. Costaban mucho más de lo presupuestado, eran irrealizables, difíciles de mantener y con un desempeño pobre. El desarrollo de software estaba en crisis. Los costos del hardware se tambaleaban mientras que los del software se incrementaban con rapidez.

Se necesitaban nuevas técnicas y métodos para controlar la complejidad inherente a los sistemas grandes.

Ingeniería de Software

El desarrollo de esta ingeniería ha mejorado considerablemente nuestro software. Comprendemos mucho mejor de las actividades involucradas en el desarrollo de software. Se han desarrollado métodos efectivos de especificación, diseño e implementación del software. Las nuevas notaciones y herramientas reducen el esfuerzo requerido para producir sistemas grandes y complejos.

Ingeniería de Software

Las preguntas que surgen cuando se construyen sistemas basados en computadora:

- ¿Por qué se requiere tanto tiempo para terminar el software?
- ¿Por qué son tan altos los costos de desarrollo?
- ¿Por qué no podemos detectar todos los errores antes de entregar el software a nuestros clientes?
- ¿Por qué dedicamos tanto tiempo y esfuerzo a mantener los programas existentes?
- ¿Por qué seguimos con dificultades para medir el avance mientras se desarrolla y mantiene el software?

Éstas y muchas otras preguntas, denotan la preocupación sobre el software y la manera en que se desarrolla, preocupación que ha llevado a la adopción de la práctica de la ingeniería del software.

¿Qué es la ingeniería de software?

- La ingeniería del software es una disciplina de la ingeniería que comprende todos los aspectos de la producción de software desde las etapas iniciales de la especificación del sistema, hasta el mantenimiento de éste después de que se utiliza.

Que no es la ingeniería de software:

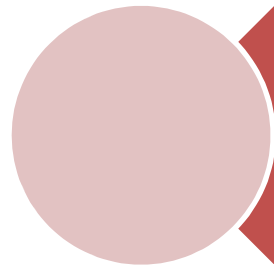
Ciencia de la computación

- **Comprende la teoría y los fundamentos.**
- La ingeniería de software comprende las formas prácticas para desarrollar y entregar un software útil.

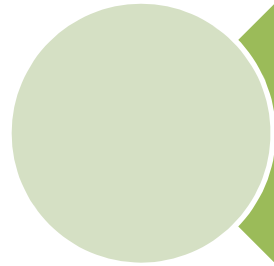
Ingeniería de sistemas

- **Se refiere a todos los aspectos del desarrollo de sistemas informáticos, incluyendo hardware, software e ingeniería de procesos.**
- La ingeniería de software es parte de este proceso

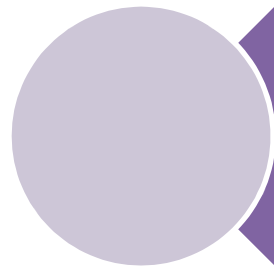
Desafíos de la ingeniería de software



Enfrentarse con la
creciente diversidad



Demandas para reducir
los tiempos de entrega



Desarrollo de software
fiable

No hay balas de plata

Frederick P. Brooks (1931-2022)

- **Complejidad accidental:** los problemas que creamos nosotros mismos y que pueden corregirse. Por ejemplo: código fuente, base de datos, estimaciones, etc.
- **Complejidad esencial:** es causada por el problema a resolver y nada puede eliminarla. (Diseño) Ejemplo: los requerimientos del cliente.

Principios de la ingeniería de software

Siete principios que se centran en la práctica de la ingeniería de software como un todo. David Hooker propuso siete principios que se centran en la práctica de la ingeniería de software como un todo.

<https://wiki.c2.com/?DavidHooker>

Primer principio: La razón por la que todo existe

- *Un sistema de software existe por una razón: **dar valor a sus usuarios**. Todas las decisiones deben tomarse teniendo esto en mente.*
- Antes de comenzar un proyecto de software, asegúrese de que el software tenga un propósito para el negocio y que los usuarios perciben valor en él.
- Preguntarse “¿Esto agrega valor real al sistema?” Si la respuesta es “no”, entonces no lo haga.
- *Todos los demás principios apoyan a éste.*

Segundo principio: MSE (Mantenlo sencillo, estúpido)

- *Todo diseño debe ser tan simple como sea posible, pero no más simple.*
- *Se requiere mucha reflexión y trabajo con iteraciones múltiples para poder simplificar.*
- *La recompensa es un software más fácil de mantener y menos propenso al error.*

Tercer principio: Mantener la visión

- *Una visión clara es esencial para el éxito de un proyecto de software.*
- *Comprometer la visión de la arquitectura de un sistema de software debilita y, finalmente hará que colapsen incluso los sistemas bien diseñados.*
- *Tener un arquitecto que pueda mantener la visión y que obligue a su cumplimiento garantiza un proyecto de software muy exitoso.*

Cuarto principio: Otros consumirán lo que usted produce

- *Siempre establezca especificaciones, diseñe e implemente con la seguridad de que alguien más tendrá que entender lo que usted haga.*
- *Hacer su trabajo más fácil agrega valor al sistema.*

Quinto principio: *Ábrase al futuro*

«Construir para hoy, diseñar para el mañana»

- ***Un sistema con larga vida útil tiene más valor.***
- *Los sistemas deben ser fáciles de adaptar a los cambios en las especificaciones, al hardware y demás cambios.*
- *Los sistemas que lo logran son los que se diseñaron para ello desde el principio. Nunca diseñe sobre algo iniciado. Siempre pregunte: “¿qué pasa si...?” y prepárese para todas las respuestas posibles mediante la creación de sistemas que resuelvan el problema general, no sólo uno específico.*

Sexto principio: Planee por anticipado la reutilización

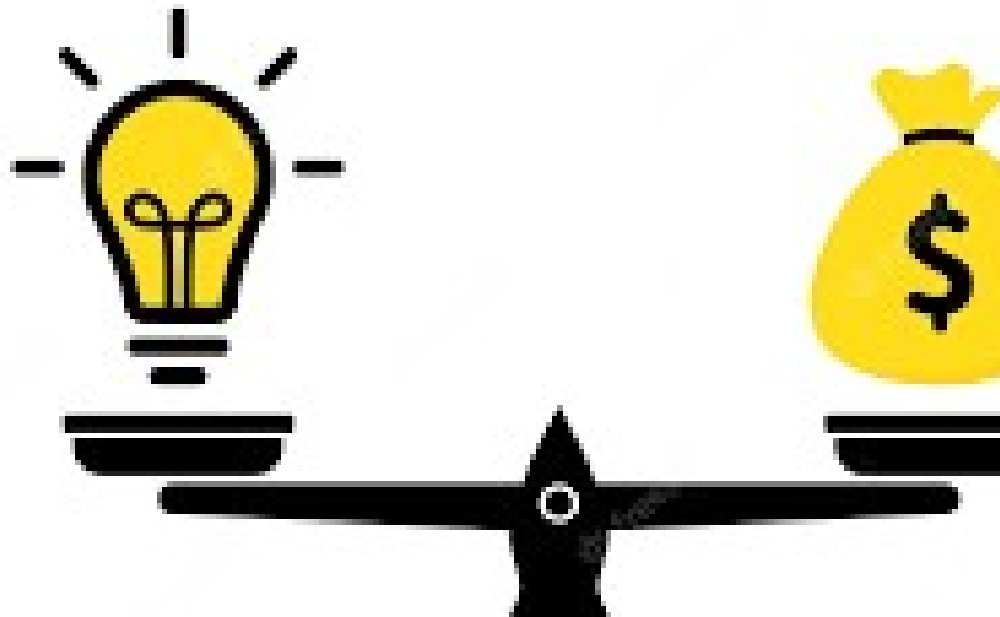
- ***La reutilización ahorra tiempo y esfuerzo.***
- *La planeación anticipada en busca de la reutilización disminuye el costo e incrementa el valor tanto de los componentes reutilizables como de los sistemas en los que se incorpora.*

Séptimo principio: ¡Piense!

- *Pensar en todo con claridad antes de emprender la acción casi siempre produce mejores resultados.*
- *Cuando en un sistema se han puesto pensamientos claros, el valor se manifiesta. La aplicación de los primeros seis principios requiere pensar con intensidad, por lo que las recompensas potenciales son enormes.*

Punto clave...

- Valuar siempre tiempo y esfuerzo (costo)



Por qué la creación de software es un problema de ingeniería.

Caso práctico – Debate.

<https://tinyurl.com/24bduhsy>

<https://forms.office.com/r/0FE9z91AZ3>

Aspectos clave de la normativa ISO 10218 (Actualización 2025):

- **ISO 10218-1:2025 (Robots):** Establece requisitos de seguridad para el diseño y fabricación de robots industriales, incluyendo nuevas clasificaciones y requisitos actualizados de seguridad funcional.
- **ISO 10218-2:2025 (Integración):** Se enfoca en la instalación, integración y salvaguarda de sistemas robóticos.
- **Colaboración Hombre-Robot (Cobots):** Consolida normativas técnicas previas (ISO/TS 15066) para la operación segura conjunta, obligando a evaluaciones de riesgos más estrictas (limitación de potencia y fuerza).
- **Ciberseguridad:** Introduce requisitos específicos de seguridad informática en robots industriales y conectados.
- **Validación:** Exige pruebas de presión obligatorias para asegurar que la **fuerza de los robots** no represente riesgo para los trabajadores

Otras normativas relacionadas:

- **ISO/TS 15066:** Guía detallada para la colaboración hombre-robot (fuerzas y presiones permitidas).
- **ISO 13849-1 / IEC 62061:** Seguridad funcional en sistemas de control.
- **Directiva de Máquinas (Unión Europea):** Marco legal obligatorio donde se armonizan las normas ISO 10218